

Python Full Stack - Module 1 Faculty Notes

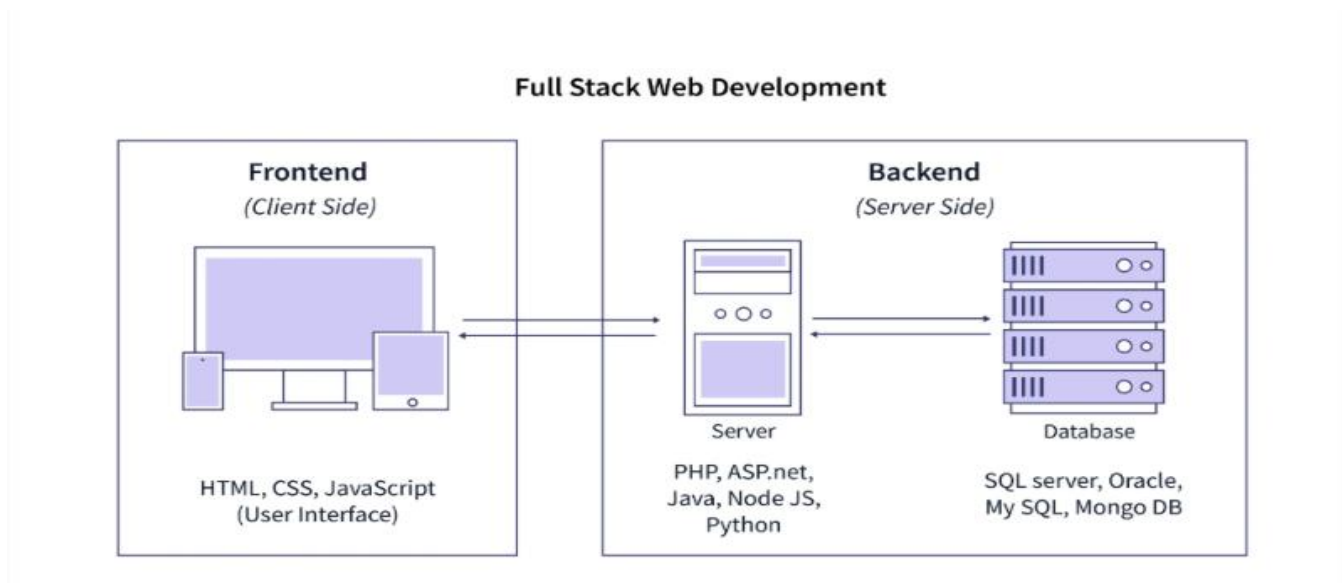
Topics: What is Full Stack Development • Frontend vs Backend vs Database • Client-Server Architecture

1. What is Full Stack Development?

Full Stack Development is the process of developing a complete web application by working on the Frontend (User Interface), Backend (Business Logic), and Database (Data Storage). A Full Stack Developer understands how these three layers communicate to deliver a complete application.

Real-world Example: When a user logs into Instagram, the browser collects the credentials (Frontend), the server validates them (Backend), the database checks the stored records (Database), and the result is returned to the browser.

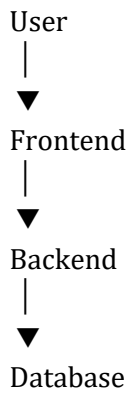
Architecture Diagram



2. Frontend vs Backend vs Database

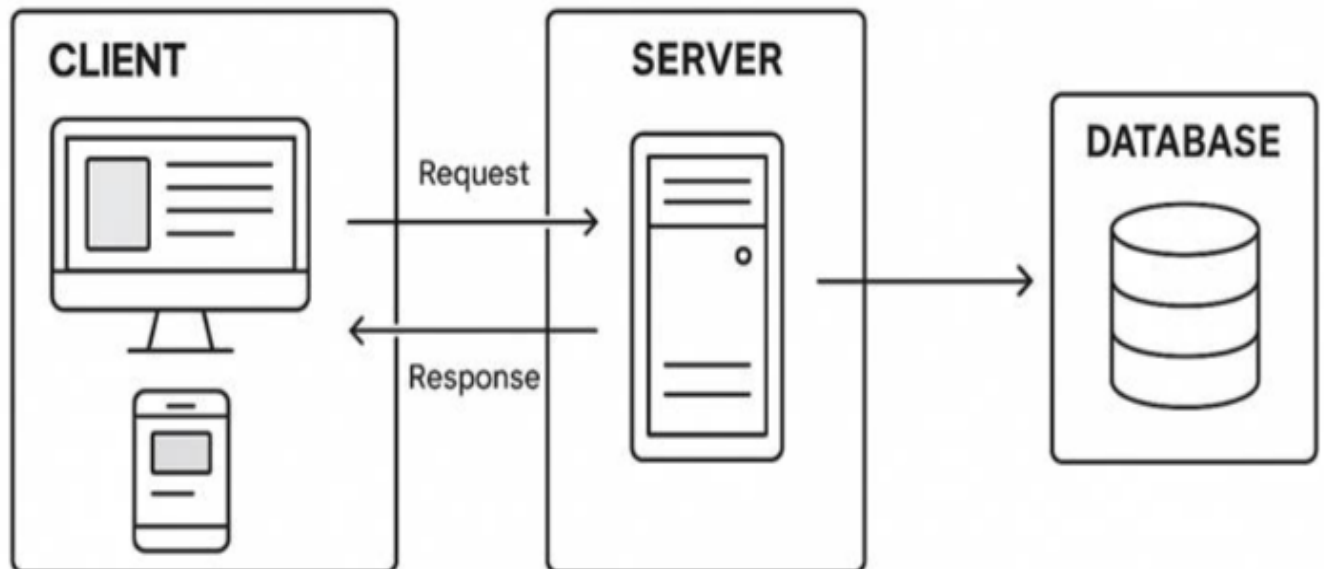
Layer	Purpose	Technologies	Responsibilities
Frontend	User Interface	HTML, CSS, JavaScript	Display UI & collect input
Backend	Business Logic	Python, FastAPI	Process requests, validation, APIs
Database	Data Storage	MySQL	Store & retrieve data

Flow:



3. Client-Server Architecture

A Client is the application used by the user (browser/mobile app). A Server receives requests, executes business logic, accesses the database if needed, and sends a response back.



What is a Client?

A **Client** is any device or application that requests information or services from a server.

Examples

- Web Browser (Chrome, Edge, Firefox)
- Mobile Apps (Instagram, WhatsApp)
- Desktop Applications

Responsibilities of a Client

- Displays the User Interface (UI)
- Takes input from the user
- Sends requests to the server
- Displays the response received from the server

What is a Server?

A **Server** is a computer or software that receives requests from clients, processes them, and sends back the required response.

Responsibilities of a Server

- Receives client requests
- Executes business logic
- Communicates with the database
- Sends the appropriate response
- Handles security and authentication

How Client-Server Architecture Works

Step 1: User Makes a Request

The user performs an action, such as clicking the **Login** button

Step 2: Client Sends Request

The browser sends an **HTTP Request** to the server.

Step 3: Server Processes Request

The server:

- Validates the data
- Executes business logic
- Connects with the database if required

Step 4: Database Returns Data

The database returns the requested information to the server.

Step 5: Server Sends Response

The server prepares an **HTTP Response** and sends it back to the client.

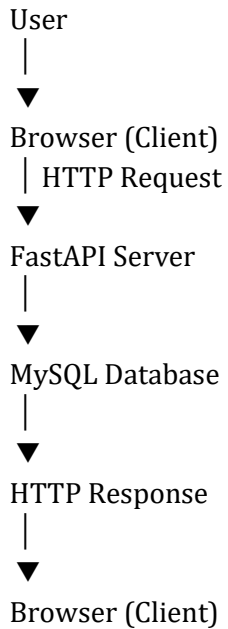
Step 6: Client Displays Result

The browser displays the result to the user.

Restaurant Analogy

- Customer → Client
- Waiter → HTTP Request
- Chef → Backend
- Kitchen Storage → Database
- Food Served → HTTP Response

Request-Response Lifecycle



1. HTTP (HyperText Transfer Protocol)

- **Definition**
HTTP is a communication protocol that allows a **client (browser)** and a **server** to exchange information over the internet.
- **Purpose**
 - Sends requests from the client to the server.
 - Receives responses from the server.
- **Features**
 - Stateless protocol (each request is independent).
 - Used for web browsing and APIs.
 - Transfers web pages, images, videos, and JSON data.
- **Example**
 - When you open www.google.com, your browser sends an **HTTP Request**, and Google's server returns an **HTTP Response** containing the webpage.

2. HTTPS (HyperText Transfer Protocol Secure)

- **Definition**
- HTTPS is the secure version of HTTP. It encrypts data before transmitting it over the internet using **SSL/TLS**.
- **Purpose**
 - Protects sensitive information.
 - Prevents data from being intercepted or modified.
 - Ensures secure communication.
- **Features**
 - Encrypts data.
 - Provides authentication.
 - Maintains data integrity.
- **Example**
- Online banking, Amazon, Flipkart, Gmail, and other secure websites use HTTPS to protect passwords and payment information.

3. TCP/IP (Transmission Control Protocol / Internet Protocol)

- **Definition**
- TCP/IP is the foundation of internet communication. It ensures that data is delivered to the correct destination accurately and reliably.
- **Components**
- **TCP (Transmission Control Protocol)**
 - Breaks data into small packets.
 - Ensures all packets arrive correctly.
 - Retransmits lost packets.
 - Reassembles packets in the correct order.
- **IP (Internet Protocol)**
 - Assigns and uses IP addresses.
 - Finds the destination device.
 - Routes packets across the internet.
- **Example**
- When downloading a large file or streaming a video, TCP ensures all data arrives correctly, while IP directs the data to your device.

4. DNS (Domain Name System)

- **Definition**
- DNS is often called the **Phonebook of the Internet**. It converts human-readable domain names into IP addresses.
- **Purpose**
 - Converts website names into IP addresses.
 - Makes websites easier to access without remembering numeric IP addresses.

- **Example**
- Instead of typing:
- 142.250.193.78
- You simply type:
- `www.google.com`
- DNS finds the corresponding IP address and connects your browser to Google's server.